



Unlocking Object Functionality in Python

Section 1: Learn

What are Objects in Python?

An **object** in Python is an **instance of a class** that contains **data (attributes)** and **behaviors (methods)**. Every entity in Python, including numbers, strings, lists, and dictionaries, is an object.

Why Use Objects?

- **Encapsulates Data and Behavior:** Objects store related data and functions together.
- **Improves Code Reusability:** Avoids repetition by defining behaviors once in a class.
- **Organizes Code Better:** Helps in designing real-world models in programming.

How Do Objects Work?

1. Define a class (blueprint for creating objects).
2. Create an object using the class.
3. Access object properties and methods.

Fun Fact: Why Were Objects Introduced?

Before Object-Oriented Programming (OOP), procedural programming was widely used, but managing large codebases was difficult. OOP simplified this by organizing code into objects!



Section 2: Practice

1. Creating and Using Objects

Defining a Class and Creating an Object

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

car1 = Car("Toyota", "Corolla")
print(car1.brand, car1.model)
```

Output:

Toyota Corolla

Problem: Create an Object for a Student

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Amit", 21)
print(student1.name, student1.age)
```

Output:

Amit 21



2. Object Methods

Adding Methods to a Class

```
class Car:

    def __init__(self, brand, model):

        self.brand = brand

        self.model = model

    def display_info(self):

        print(f"Car: {self.brand} {self.model}")

car1 = Car("Honda", "Civic")

car1.display_info()
```

Output:

```
Car: Honda Civic
```

Problem: Add a Method to Calculate Student Age in 5 Years

```
class Student:

    def __init__(self, name, age):

        self.name = name

        self.age = age

    def future_age(self, years):

        return self.age + years
```



```
student1 = Student("Amit", 21)
print("Age in 5 years:", student1.future_age(5))
```

Output:

```
Age in 5 years: 26
```

3. Working with `__str__()`

`__str__()` for Readable Output

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"{self.title} by {self.author}"

book1 = Book("Python Basics", "John Doe")
print(book1)
```

Output:

```
Python Basics by John Doe
```

Problem: Add `__str__()` to Student Class

```
class Student:
    def __init__(self, name, age):
```



```
self.name = name

self.age = age

def __str__(self):
    return f"Student Name: {self.name}, Age: {self.age}"

student1 = Student("Amit", 21)
print(student1)
```

Output:

```
Student Name: Amit, Age: 21
```

4. Class vs Instance Variables

Example of Instance Variables

```
class Laptop:

    def __init__(self, brand, price):
        self.brand = brand # Instance variable
        self.price = price # Instance variable

laptop1 = Laptop("Dell", 50000)
laptop2 = Laptop("HP", 60000)

print(laptop1.brand, laptop1.price) # Dell 50000
print(laptop2.brand, laptop2.price) # HP 60000
```



Example of Class Variable

```
class Laptop:
    discount = 10 # Class variable

    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

print(Laptop.discount) # Accessing class variable
```

Output:

```
10
```

5. Inheritance: Reusing Functionality

Creating a Child Class

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return "Some sound"

class Dog(Animal):
    def make_sound(self):
        return "Bark!"
```



```
dog1 = Dog("Buddy")  
print(dog1.name, "-", dog1.make_sound())
```

Output:

Buddy - Bark!

Problem: Create a **Bike** Class That Inherits from **Vehicle**

```
class Vehicle:  
    def __init__(self, brand):  
        self.brand = brand  
  
class Bike(Vehicle):  
    def ride(self):  
        return f"Riding a {self.brand} bike!"  
  
bike1 = Bike("Yamaha")  
print(bike1.ride())
```

Output:

Riding a Yamaha bike!

Try This: Check Object Type with **isinstance()**

```
car = Car("Ford", "Mustang")  
print(isinstance(car, Car)) # True
```



```
print(isinstance(car, Student)) # False
```

Output:

True

False

Section 3: Know More (FAQs)

Q1: What is the difference between a class and an object?

Concept	Definition
Class	A blueprint for creating objects
Object	An instance of a class

Example:

```
class Car:
```

```
    pass
```

```
car1 = Car() # Object of class Car
```

Q2: Can an object have multiple methods?

Answer: Yes, objects can have multiple methods.

Example:

```
class Calculator:
```

```
    def add(self, a, b):
```



```
return a + b
```

```
def multiply(self, a, b):
```

```
    return a * b
```

```
calc = Calculator()
```

```
print(calc.add(5, 3))
```

```
print(calc.multiply(5, 3))
```

Q3: How do I check if an object has an attribute?

Answer: Use `hasattr()`.

Example:

```
print(hasattr(car1, "brand")) # True
```

```
print(hasattr(car1, "color")) # False
```

Q4: How to delete an object attribute?

Answer: Use `del`.

Example:

```
del car1.brand # Removes the brand attribute
```

Q5: What happens if I print an object without `__str__()`?

Answer: Python prints the object's memory location.

Example:

```
class Example:
```

```
    pass
```



```
obj = Example()  
print(obj)
```

Output:

```
<__main__.Example object at 0x12345678>
```

Objects make Python more structured and reusable. 🚀 Keep practicing to master real-world applications of OOP!